COMPUTER SCIENCE + COMPUTER ENGINEERING

06

Module 22: Explicit GPU programming and graphics APIs

Expanded Volume 05 / four-lesson module

4 lessons · 20 lesson questions · 5 review questions · Integrating project

PACKAGE 06 / EXPANDED VOLUME 05 / SEPTEMBER 28, 2026
Self-study: derive, implement, test and explain.

Module 22 — Explicit GPU programming and graphics APIs

Prerequisites: Module 21 plus Modules 18 and 20. Choose one compatible backend, not all at once.

Planning allocation: 90-140 planning hours, including the project's work and review.

Lesson	Outcome
22.1 — GPU work submission, resources and pipeline state	Separate CPU recording from asynchronous GPU execution.
22.2 — Buffers, shaders, layouts and resource binding	Match data representations across CPU and GPU boundaries.
22.3 — Synchronization, hazards and frames in flight	Express dependencies without serializing the entire renderer.
22.4 — Presentation, resizing and device capability policy	Treat the window system as a changing environment rather than a fixed image.

Original module project

Implement a GPU backend for the reference scene. Query device capabilities, enable available validation, document the toolchain and handle resize and shutdown safely. A CPU backend remains a valid fallback while hardware access is arranged.

Integrating milestones

1. P22-A: capability report, clear target and validation capture.
2. P22-B: geometry, transform layout and bounded compute readback.
3. P22-C: compute-to-draw dependency and two reusable contexts.
4. P22-D: resizing, loss policy and failure-safe teardown.

Evidence to retain

- Actual backend/toolchain record; byte layout table; shader sources; readback results.
- Dependency ledger; frame-context stress log; resize/minimize matrix; pending hardware markers where applicable.

Read the four lessons, keep the assigned readings, implement the workbook labs and use the module review before advancing. The reference code is a diagnostic aid; milestone completion is learner work.

LESSON 22.1 / MODULE 22

GPU work submission, resources and pipeline state

> **Learning outcome:** Separate CPU recording from asynchronous GPU execution.

Two processors, several timelines

A graphics program does not call a function and receive a finished image. The CPU constructs resources and records commands, then submits work that the GPU executes asynchronously. Separate the application timeline, command-recording lifetime, queue submission and completion. The return from a submit call is not evidence that a buffer is no longer in use.

Describe an initial frame as acquire target, record clear/draw operations, submit, present and eventually observe completion. Some APIs expose image acquisition directly; browser WebGPU manages more of the presentation machinery. The conceptual distinction still matters. A frame can be recorded but unsubmitted, submitted but incomplete, or complete but not yet visibly presented. An event loop must also remain responsive while these states change. [S05, S06]

Objects have different jobs

The device represents a selected implementation and enabled capability set. Buffers and textures provide storage. Views describe how storage is interpreted. A pipeline packages compatible shader stages and fixed-function state. Bindings connect shader-visible slots to particular resources and ranges. Command encoders or command buffers record operations, while queues execute submissions. A window or canvas surface is an output interface, not the device itself.

Ownership follows the asynchronous consumer. A CPU object going out of scope may destroy its wrapper while GPU commands still reference the underlying allocation. In native APIs, arrange deferred destruction after the relevant completion point. In managed APIs, follow their explicit object validity and destruction rules; do not infer the exact native lifetime contract from JavaScript garbage collection. The package's ownership ledger models completion serials on the CPU; it cannot validate a driver. [S06, S09]

Choose one backend deliberately

The canonical routes are Vulkan, Direct3D 12 or Metal on a compatible platform. The included WebGPU example is an additional teaching bridge for command recording, shaders, resource binding and bounded compute; it is not a substitute for learning native barriers when native engine work is the target. Select one route for the module project. Do not spend the volume implementing every API abstraction before you can show a validated clear operation.

Record OS, adapter, driver where exposed, SDK/browser build, shader compiler and enabled features. Query rather than assume limits and formats. An integrated GPU can be sufficient for these small labs; no performance claim follows merely from a marketing name. A software-only route remains valid while access is unavailable, but its report must say GPU validation pending.

Start with observable failure

Create a labeled device and target, enable available validation, and capture errors in a file or a visible log. Then clear to an unmistakable color. Check creation failures, a missing adapter and shutdown before adding

geometry. The useful first artifact is a capability report plus a correctly attributed clear operation, not a complex image with unexplained warnings.

The WebGPU example intentionally uses no optional advanced feature. It reports its actual device limits and failure messages when run. It is supplied as source; the package's automated CPU checks do not turn that source into hardware-tested evidence. This distinction is part of professional engineering, not a reason to skip the experiment.

Worked example

Suppose the CPU finishes recording at 1.2 ms, submit returns at 1.3 ms, the GPU starts at 1.5 ms and finishes at 5.8 ms. The CPU recording cost is 1.2 ms under that timer scope. Submission latency is roughly 0.1 ms. GPU execution is 4.3 ms only if those GPU timestamps were actually measured on a compatible clock. A CPU timer around submit cannot establish the 4.3 ms number.

A vertex buffer referenced by that submission may not be overwritten or freed merely because submit returned at 1.3 ms. Its last-use token corresponds to completion at 5.8 ms in this hypothetical timeline. A screenshot appearing at 16.7 ms would include presentation scheduling as well, and does not make the GPU work take 16.7 ms.

Write a lifetime ledger with columns resource, owner, last submission, completion test and retirement action. Populate it for the target, vertex buffer, uniform buffer and command storage before the first animated frame.

Original lab assignment

Create a window or offscreen target and report supported capabilities. Pass: render a clear color, capture validation output and explain which objects belong to CPU recording versus GPU execution.

Assigned study

R21: Introduction and Drawing a Triangle; alternatively R24 or R25 for one native platform; R22: SDL3 platform setup.

Focused study: S05: frames in flight. S06: device, encoding and queue mental model. R21/R24/R25: one selected native backend setup.

Check your understanding

1. Does queue submission mean the GPU has finished?
2. Why can immediate buffer destruction be unsafe?
3. Must a beginner implement three graphics APIs simultaneously?
4. A submit call returns after 0.2 ms. Can you conclude the GPU rendered in 0.2 ms? Explain.
5. A requested optional feature is absent. Give two valid policy responses and one invalid response.

LESSON 22.2 / MODULE 22

Buffers, shaders, layouts and resource binding

> **Learning outcome:** Match data representations across CPU and GPU boundaries.

Host and shader share bytes, not field names

A shader reads a buffer according to its own layout rules. A C++ struct with identically named members can have different offsets, padding, array strides or matrix interpretation. Write the byte contract before uploading. Distinguish vertex attribute layout, storage-buffer layout and uniform-buffer layout: they are separate contracts even inside the same API.

For the browser example, vertex data is two vec4 values: position at byte 0 and linear color at byte 16, giving a 32-byte vertex stride. The transform uniform is a column-major mat4x4, occupying 64 bytes. In C++, static_assert and offsetof can check the matching host representation; in JavaScript, explicitly index the Float32Array. A correct size with a transposed matrix still produces wrong geometry. [S01]

A particularly useful WGSL example is a struct containing position:vec3 followed by scale:f32. The vector has alignment 16 and size 12, so scale may occupy offset 12 and the struct can occupy 16 bytes. But an array of vec3 has a 16-byte element stride. Do not summarize this as either all vec3 values are 12 bytes or all are 16 bytes. Consult the rule for the actual address space and aggregate. Dynamic binding offsets have an additional device-limit alignment requirement. [S01]

Bindings are a resource interface

A resource binding associates a shader location with a buffer or texture and a permitted range or view. Pipeline layouts constrain type, visibility and compatibility. Binding the right data to the wrong group or slot is not repaired by renaming a variable. Treat binding identifiers as part of the interface version and keep a table beside the shader.

The upload path must also state whether the API copies source bytes immediately, stages them, or permits persistent mapping under specified synchronization. Do not mutate CPU staging storage on the assumption that every API behaves like the one used in a previous lesson. A mapped GPU buffer generally needs a deliberate ownership transition before execution can consume it. [S06]

Compute covers work in groups

For N elements and group size G, launch ceil(N/G) groups. Global invocation i operates only when i<N. For N=1000 and G=64, sixteen groups create 1,024 invocations, so the last 24 must do no element access. Reject invalid sizes and query the selected device's limits rather than assuming any group shape is portable.

Use a simple bounded function, such as output[i]=2*input[i]+1, and read the result back. A visible triangle alone does not check compute correctness. For float comparisons, choose an expected tolerance based on the operations; for exactly representable small integers in f32, exact comparison can be appropriate. Check N=0, 1, G-1, G and G+1. Zero work should be handled explicitly rather than creating a zero-size resource and hoping the API accepts it.

Build a diagnostic ladder

Test raw bytes, then a known transform, then resource bindings, then readback. Only after these work should compute write data consumed by a draw. The included browser demonstration computes vertex output and

draws from that storage, with a separate readback check. Hardware execution remains a learner validation step.

Worked example

For a position/scale storage record, the chosen WGSL layout is position at 0, scale at 12, total 16. Two records occupy 32 bytes. A different record with position:vec3 followed by color:vec3 places color at offset 16 and has total size 32 because the aggregate is rounded to its 16-byte alignment.

For 1,000 elements and 64 threads per group, (1000+63)//64=16. The shader guard is if (i >= count) { return; }. A test with 1,025 elements would need 17 groups, making off-by-one assumptions visible.

For a column-major translation matrix, place tx,ty,tz in the first three elements of the last column. Multiplying the homogeneous point (0,0,0,1) must produce (tx,ty,tz,1). Test this special point before testing a rotating object, where a layout error is harder to diagnose.

Original lab assignment

Upload a transform and vertex colors, then add a bounded compute operation. Pass: host/shader layouts are documented, a readback test checks results, and non-multiple work sizes remain correct.

Assigned study

R21: vertex buffers, uniform buffers and compute; R37: execution configuration and memory organization, only for the optional CUDA route.

Focused study: S01: alignment, size, structure and array layout; compute built-ins. S06: buffer mapping and bindings. R21: upload and compute route.

Check your understanding

1. Why can matching field names still produce a layout bug?
2. How many 64-thread groups cover 1,000 elements?
3. What does a resource binding establish?
4. A WGSL storage struct contains vec3 position and f32 scale. What are the member offsets and total size?
5. How many 64-thread groups cover 1,025 elements, and why is a bounds guard still required?

LESSON 22.3 / MODULE 22

Synchronization, hazards and frames in flight

> **Learning outcome:** Express dependencies without serializing the entire renderer.

Name the dependency before choosing a primitive

A producer writes data that a consumer later reads: this is a read-after-write hazard. Execution order is necessary, but some APIs also require an explicit memory dependency making those writes available and visible in the consumer's access domain. A write-after-read dependency needs the old read to finish before new writes; a write-after-write dependency must order the writes appropriately. Start with resource, range, producer stage/access and consumer stage/access. Only then select the API mechanism. [S07]

In a Vulkan single-queue example, compute storage writes consumed as vertex input require a dependency from compute-shader writes to vertex-input reads of the correct range. A color attachment written by a render pass and sampled in a later fragment shader needs different stages and access masks, and possibly an image layout transition. A shader-stage barrier cannot be guessed from the word texture. Cross-queue work also requires appropriate submission synchronization and, where applicable, queue-family ownership transfer.

Direct3D 12 exposes resource states and barriers with its own rules. Metal has resource tracking and synchronization modes that must be read in their native context. WebGPU validates usage scopes and manages many transitions automatically; sequencing separate legal compute and render passes is not evidence that a native barrier can be omitted. The abstract dependency is shared; the exact API expression is not. [S09, S10, S06]

Frames in flight need reusable ownership units

A frame context groups mutable resources that will be reused together: command storage, per-frame uniform storage and relevant temporary allocations. Give each context a completion token. Before resetting or overwriting context k, establish that the submission last using k has completed. Two contexts allow useful overlap without treating the entire renderer as globally idle after every operation. Static read-only resources need not all be duplicated.

Frame index and acquired presentation-image index are different. In Vulkan, presentation's wait on a render-finished binary semaphore can outlive the graphics submission fence. A fence for a frame is therefore not, by itself, proof that its present-wait semaphore is reusable. Follow a supported lifetime scheme, such as render-finished semaphores indexed by acquired swapchain image, and consult the relevant presentation-completion rules. This is a common bug in otherwise plausible two-frame tutorials. [S05, S08]

Completion controls retirement

Suppose a texture's last use is submission 12. Replacing its handle at submission 13 does not immediately free the old allocation. Queue it for retirement and release it when the completion ledger includes 12, with every queue that used it accounted for. The monotonic serial model in this package is deliberately single-queue. A multi-queue renderer needs a completion set or equivalent dependency proof, not a single global number invented by the CPU.

Stress the boundary

Slow the GPU path or increase work so premature reuse becomes visible. Alternate contexts, resize, reload a resource and exit while work is outstanding. Validation is necessary but not a complete proof; some logical

lifetime bugs produce legal commands using wrong data. Pair validation logs with frame IDs encoded into output and readback where practical. Use coarse waiting for initial correctness and controlled teardown, then replace unnecessary waits with precise dependencies.

Worked example

Context A was last submitted as serial 10, context B as 11, and the completed serial is 9. Neither can be reset. Once completion advances to 10, A may be reused for serial 12 while B remains busy. Merely calculating frame_number % 2 does not prove the chosen context is free.

For a compute-to-vertex-buffer path, the dependency ledger reads: resource generated_vertices; producer compute shader / storage write; consumer vertex input / attribute read; same queue; buffer range [0,vertex_bytes). A shader-storage read in a vertex shader would be a different consumer access than fixed-function vertex input. Name the actual consumption mechanism.

If three swapchain images coexist with two CPU frame contexts, valid image indices are 0,1,2 while context indices are 0,1. An image-indexed array of length two is an immediate bounds bug, and a frame-indexed present semaphore can have a subtler reuse bug even when every index stays in range.

Original lab assignment

Implement a compute-to-draw dependency and two reusable frame contexts. Pass: validation reports no relevant hazards and a stress test never resets a context still used by the GPU.

Assigned study

R21: synchronization, frames in flight and resource transitions; use R24 or R25 equivalents for the selected backend.

Focused study: S07: compute/graphics dependencies. S05: frame contexts. S08: swapchain semaphore reuse. S09/S10: native backend equivalents.

Check your understanding

1. Are execution ordering and memory visibility identical?
2. When may a frame context be reused?
3. Why is waiting for the whole GPU after each command a poor final design?
4. Context A last used serial 10 and completed serial is 9. Is frame_number modulo two sufficient to reuse A?
5. Why can a graphics submission fence be insufficient to prove that a Vulkan presentation-wait semaphore is reusable?

LESSON 22.4 / MODULE 22

Presentation, resizing and device capability policy

> **Learning outcome:** Treat the window system as a changing environment rather than a fixed image.

The drawable is changing state

A window can be resized, minimized, moved between displays or closed during initialization. Its logical size may differ from its framebuffer pixel size. Query the drawable size in pixels for render-target allocation and aspect ratio; do not assume a high-density display uses one pixel per window-coordinate unit. A zero width or height is a valid state to handle, not a valid render-target extent for the normal path. [S11]

Write a surface state machine: unavailable, ready, resize pending, rebuilding, lost and stopping. Rendering only consumes a ready surface. A resize event records the new desired extent; rebuild at a controlled boundary after handling resources still in flight. Do not rebuild independently inside an event callback that can race the render path.

Recreate a dependency set, not just a viewport

Size-dependent resources may include depth, multisample color, post-processing targets, history buffers, framebuffers and projection data. Temporal history often becomes invalid when size or projection changes. Build the new set transactionally: validate extents, allocate compatible resources, then publish it. Retire old resources only after their last use. Under allocation failure, preserve a known valid state or stop with an explanation.

In Vulkan, handle out-of-date and suboptimal results according to policy, including acquisition and presentation paths. A common deadlock arises when a fence is reset and acquisition then returns early without a submission that will signal it. Delay reset until the code is committed to submission, or otherwise repair the state. A minimal renderer may wait for idle during recreation; make that deliberate, not a per-frame habit. [S12]

Capabilities become product policy

Divide features into required, optional and disabled. Required features gate startup. Optional features select a tested fallback when absent. Disabled features are excluded even if the adapter advertises them, because this build has no validated implementation. Record enabled device capabilities rather than merely the maximum capabilities advertised by the adapter.

Shader compilation and pipeline creation can fail. Device loss is a distinct condition from window resize: a lost device may invalidate the entire device-owned resource set. The included browser bridge stops, reports loss and requires reinitialization through a fresh page run; it does not claim seamless recovery. An engine may choose a controlled restart rather than implementing transparent recreation of every subsystem.

Teardown is an ordinary execution path

Stop scheduling new work, cancel pending loads, wait or otherwise satisfy the required completion conditions, release resources in an ownership-safe order, and close platform objects. Test shutdown after each initialization stage, not only after a successful frame. Use an initialization record or RAII owners so cleanup acts on exactly the objects that exist.

Your compatibility matrix is evidence, not a wish list. Record actual runs separately from plausible support. A software fallback with honest limitations is more useful than a GPU path whose warnings and failures have been edited out of the report.

Worked example

A window reports logical size 800-by-600 but drawable size 1600-by-1200. A depth target allocated at logical resolution would not match a full-resolution color target. At four bytes per depth sample, the correct single-sample depth allocation is 1600*1200*4=7,680,000 payload bytes, before API alignment and metadata.

A stress sequence is (1280,720), (0,0), (800,600), (1600,1200), close. The zero extent should suspend size-dependent rendering without dividing by height or allocating an ordinary zero-size target. Restoration invalidates size-dependent history and creates the new target set.

In the early-return fence bug, wait succeeds, reset makes the fence unsignaled, acquisition returns out-of-date, and the function returns. The next wait can hang forever because no submission was issued to signal that fence. Draw the state transition to explain why the reset belongs after a successful acquisition decision.

Original lab assignment

Stress resize, minimize, restore and orderly shutdown. Pass: no stale size-dependent resources, no division by zero, and a readable capability report explains enabled features and fallbacks.

Assigned study

R21: swapchain recreation and device selection; R22: window events; R24 or R25: presentation and resource lifetime.

Focused study: S11: pixel drawable size. S12: swapchain recreation and fence-reset placement. S06: device loss and canvas lifecycle.

Check your understanding

1. Why can resizing require more than changing the viewport?
2. What should happen for a zero-height drawable?
3. What is the purpose of an explicit feature policy?
4. An 800-by-600 logical window has scale factor two in both dimensions. What drawable extent and four-byte depth payload result?
5. Explain how resetting a fence before an acquire-out-of-date early return can deadlock the next frame.

Module 22 review

1. Why can correct submission order still be an incomplete native synchronization argument?
2. Are WebGPU implicit transitions proof that Vulkan barriers are unnecessary?
3. What separates frame-context indices from presentation-image indices?
4. Why should GPU results include readback instead of only a triangle?
5. What can a CPU-only run honestly establish for this module?

Project handoff

Retain the one validated gpu route evidence. Explain one failure you deliberately induced and why the fix addresses its cause. Resolve review gaps before treating this module's outputs as reliable inputs to the next one.

Reading references: IDs retain their meanings in source/REFERENCES.md and the full handbook. Use those links alongside the lesson-specific reading assignments.